docker-compose.yml is like a recipe file that tells Docker "start these 5 services for me":

Service	What it is	Simple analogy
Zookeeper	Manager for Kafka	Security guard who keeps track of everything
Kafka	Message receiver	A post office that receives all incoming mail
RabbitMQ	Message deliverer	Sorting office with URGENT and NORMAL trays
MongoDB	Database	Notebook where routing rules are stored
Redis	Fast memory cache	Post-it note on the desk — faster than opening the notebook

```
thanujasekuri@Thanujas-MacBook-Air pipeline-orchestrator % docker
compose up -d
[+] Running 6/6
 ✔ Network pipeline-orchestrator_default  Created
0.0s
 ✔ Container rabbitmq                     Started
0.4s
 ✔ Container redis                        Started
0.3s
 ✔ Container mongodb                      Started
0.3s
 ✔ Container zookeeper                    Started
0.3s
 ✔ Container kafka                        Started
0.4s
thanujasekuri@Thanujas-MacBook-Air pipeline-orchestrator % docker ps
CONTAINER ID   IMAGE                                      COMMAND
CREATED             STATUS                  PORTS
NAMES
81f2eef9ee30   confluentinc/cp-kafka:7.5.0                "/etc/confluent/
dock..."   About a minute ago   Up About a minute        0.0.0.0:9092->9092/tcp
kafka
e7b5d4071214   confluentinc/cp-zookeeper:7.5.0            "/etc/
confluent/dock..."   About a minute ago   Up About a minute        2888/tcp,
0.0.0.0:2181->2181/tcp, 3888/tcp
zookeeper
```

```
a3b71ddb626c   redis:7-alpine                                  "docker-entrypoint.s..."   About a minute ago   Up About a minute        0.0.0.0:6379->6379/tcp                                  redis
557ee358c876   rabbitmq:3-management                           "docker-entrypoint.s..."   About a minute ago   Up About a minute        4369/tcp, 5671/tcp, 0.0.0.0:5672->5672/tcp, 15671/tcp, 15691-15692/tcp, 25672/tcp, 0.0.0.0:15672->15672/tcp   rabbitmq
606bbc28843f   mongo:7                                         "docker-entrypoint.s..."   About a minute ago   Up About a minute        0.0.0.0:27017->27017/tcp                                mongodb
6db7863767d7   bde2020/hadoop-historyserver:2.0.0-hadoop3.2.1-java8     "/entrypoint.sh /run..."   2 months ago         Up 12 minutes (healthy)   0.0.0.0:8188->8188/tcp                                  historyserver
9d816b98bfda   bde2020/hadoop-nodemanager:2.0.0-hadoop3.2.1-java8       "/entrypoint.sh /run..."   2 months ago         Up 12 minutes (healthy)   0.0.0.0:8042->8042/tcp                                  nodemanager
f4fcb8d94c54   bde2020/hadoop-resourcemanager:2.0.0-hadoop3.2.1-java8   "/entrypoint.sh /run..."   2 months ago         Up 12 minutes (healthy)   0.0.0.0:8088->8088/tcp                                  resourcemanager
8ddb9a430ab4   bde2020/hadoop-datanode:2.0.0-hadoop3.2.1-java8         "/entrypoint.sh /run..."   2 months ago         Up 12 minutes (healthy)   0.0.0.0:9864->9864/tcp                                  datanode
668077a78c7b   bde2020/hadoop-namenode:2.0.0-hadoop3.2.1-java8         "/entrypoint.sh /run..."   2 months ago         Up 12 minutes (healthy)   0.0.0.0:9000->9000/tcp, 0.0.0.0:9870->9870/tcp          namenode
e44aaedfedb2   postgres:15                                     "docker-entrypoint.s..."   4 months ago         Up 12 minutes            0.0.0.0:5432->5432/tcp                                  xeno_postgres
thanujasekuri@Thanujas-MacBook-Air pipeline-orchestrator %
```

Week 1 — Detailed Progress Report

🎯 Goal

Get all infrastructure services running locally and prove a message can flow through Kafka end to end.

✅ What Was Done

1. Project Folder Created

- Created pipeline-orchestrator/ as the root project directory
- Opened in VS Code as the working environment

2. Docker Compose Setup

- Created docker-compose.yml defining all 5 services
- Used confluentinc/cp-kafka:7.5.0 and confluentinc/cp-zookeeper:7.5.0 images (more stable than Bitnami on Mac)
- All 5 containers started successfully with docker compose up -d

3. Services Verified Running

Service	Image	Port	Status
Zookeeper	confluentinc/cp-zookeeper:7.5.0	2181	✅ Up
Kafka	confluentinc/cp-kafka:7.5.0	9092	✅ Up
RabbitMQ	rabbitmq:3-management	5672, 15672	✅ Up
MongoDB	mongo:7	27017	✅ Up
Redis	redis:7-alpine	6379	✅ Up

4. RabbitMQ Dashboard

- Accessible at http://localhost:15672
- Login with admin / password confirmed working ✅

5. TypeScript Project Initialized

- npm init -y — created package.json
- Installed runtime dependencies: kafkajs, amqplib, mongoose, redis, express, cors
- Installed dev dependencies: typescript, ts-node, @types/node, @types/amqplib, @types/express, @types/cors
- npx tsc --init — created tsconfig.json
- Configured tsconfig.json with strict mode, ES2020 target, commonjs module

6. Folder Structure Created

```
pipeline-orchestrator/
├── docker-compose.yml
├── package.json
├── tsconfig.json
└── src/
    ├── config/
    |   └── index.ts        ✅ done
    ├── models/
    ├── db/
    ├── ruleEngine/
    ├── orchestrator/
    ├── consumers/
    |   └── testConsumer.ts ✅ done
    ├── producers/
    |   └── testProducer.ts ✅ done
    └── api/
```

7. Config Module

- Created src/config/index.ts

- Exports all connection strings and settings in one place — Kafka brokers, RabbitMQ URL, MongoDB URI, Redis URL, API port, cache TTL, max retries

8. Kafka Producer & Consumer

- src/producers/testProducer.ts — connects to Kafka and sends a test payment message
- src/consumers/testConsumer.ts — connects to Kafka and logs every received message
- Both tested and working ✅

9. End-to-End Kafka Flow Confirmed

- Message sent by producer:

id: 'msg-001'
type: 'payment'
priority: 'critical'
amount: 75000

- Same message received and logged by consumer ✅
- Kafka leadership election completed successfully
- No errors, no message loss

🧠 Key Understanding

Concept	What you learned
Docker Compose	One file, one command, 5 services running
Kafka	Receives messages from producers, holds them, delivers to consumers
Zookeeper	Kafka's internal manager — handles leadership and coordination
TypeScript project	Strict typing catches errors before runtime
Config module	All settings in one place — no hardcoded values anywhere else

⚠️ Issues Faced & Fixed

Issue	Fix
bitnami/zookeeper:3.9.2 image not found on Docker Hub	Switched to confluentinc/cp-zookeeper:7.5.0 and confluentinc/cp-kafka:7.5.0 — more stable and actively maintained
version attribute warning in docker-compose.yml	Removed the version field — it's obsolete in newer Docker versions

Kafka leadership election error on first consumer start	Normal behaviour — Kafka needs a few seconds to elect a partition leader. Resolved itself automatically

📦 Files Created This Week

File	Purpose
docker-compose.yml	Defines and starts all 5 infrastructure services
package.json	Node.js project manifest with all dependencies
tsconfig.json	TypeScript compiler configuration
src/config/index.ts	Central config — all connection strings and settings
src/producers/testProducer.ts	Sends a test message into Kafka
src/consumers/testConsumer.ts	Reads messages from Kafka and logs them

Week 2 — Detailed Progress Report

🎯 Goal

Push all code to GitHub and set up an automated CI pipeline that runs on every code push.

✅ What Was Done

1. Git Repository Initialized

- Ran git init inside pipeline-orchestrator/ folder
- Created .gitignore to exclude node_modules/, dist/, .env and map files from being pushed
- Made first commit with all Week 1 files:

8 files changed, 1703 insertions
.gitignore
docker-compose.yml
package.json
package-lock.json
tsconfig.json
src/config/index.ts
src/consumers/testConsumer.ts
src/producers/testProducer.ts

2. GitHub Repository Created

- Repo created at https://github.com/thanujaa9/pipeline-orchestrator
- Set to **Public** so examiners can view it

- Connected local repo to GitHub remote with git remote add origin
- Pushed code with git push -u origin main
- All 14 objects uploaded successfully ✅

3. GitHub Actions CI Pipeline

- Created .github/workflows/ci.yml
- Pipeline triggers automatically on every push to main branch
- Pipeline triggers on every pull request to main branch

Pipeline steps configured:

Step	What it does
Checkout code	Downloads your code onto the CI server
Setup Node.js	Installs Node.js v18 on the CI server
Install dependencies	Runs npm install to get all packages
TypeScript build check	Runs npx tsc --noEmit to check for type errors

4. CI Pipeline Result

- Pushed commit: "Week 2: Add GitHub Actions CI pipeline"
- Pipeline ran automatically within seconds
- All steps passed ✅
- Total build time: **16 seconds**
- Status: **Green** ✅

🧠 Key Understanding

Concept	What you learned
Git	Tracks every change you make to your code with a message explaining what changed
GitHub	Cloud storage for your code — accessible by your whole team and examiners
.gitignore	Tells Git to ignore node_modules/ — no point uploading 1700 files that can be reinstalled with one command
GitHub Actions	A robot that automatically checks your code every time you push — catches errors before they become problems

tsc --noEmit	Compiles TypeScript to check for errors but doesn't produce output files — just validates the code

⚠️ Issues Faced & Fixed

Issue	Fix
Git committer showed Thanuja Sekuri instead of Christo Tonio	Set git config --global user.name and user.email to correct identity for future commits
GitHub rejected password during push	GitHub no longer accepts passwords — used Personal Access Token instead

📦 Files Created This Week

File	Purpose
.gitignore	Excludes node_modules, dist, .env from Git
.github/workflows/ci.yml	GitHub Actions CI pipeline — runs TypeScript check on every push

📊 Repository Status

- **URL:** https://github.com/thanujaa9/pipeline-orchestrator
- **Branch:** main
- **CI Status:** ✅ Passing
- **Total files on GitHub:** 9 files

Week 3 — Detailed Progress Report

🎯 Goal

Define TypeScript data models, set up MongoDB and Redis connections, and seed the 5 sample routing rules into the database.

✅ What Was Done

1. Message Model

- Created src/models/message.model.ts
- Defines the exact structure of every message flowing through the system
- Uses TypeScript strict typing — priority is a union type, optional fields marked with ?

id, timestamp, type, priority, amount?, riskScore?, kafkaTimestamp?, payload

2. Rule Model

- Created src/models/rule.model.ts
- Defines the structure of every routing rule stored in MongoDB
- operator restricted to exactly 4 values: equals, contains, greaterThan, lessThan
- targetQueue restricted to exactly 2 values: urgent, batch

3. MongoDB Connection Module

- Created src/db/mongo.ts
- Defines two Mongoose schemas matching the technical specification exactly:
  - RuleSchema — for routing rules collection
  - RoutingHistorySchema — for audit log of every routing decision
- Exports RuleModel and RoutingHistoryModel for use by Rule Engine, Orchestrator, and API
- Exports connectMongo() function — connects once on startup, throws on failure

4. Redis Connection Module

- Created src/db/redis.ts
- Exports redisClient instance used by the Rule Engine
- Exports connectRedis() function — must be called before any cache operations
- Throws on failure so calling service halts cleanly

5. Database Seeding Script

- Created src/db/seedRules.ts
- Connects to MongoDB, clears existing rules, inserts 5 fresh sample rules
- Disconnects cleanly using mongoose.disconnect() after write is confirmed
- All 5 rules verified in MongoDB ✅

6. 5 Sample Rules Seeded

Priority	Field	Operator	Value	Target Queue
1	priority	equals	critical	urgent
2	amount	greaterThan	10000	urgent
3	riskScore	greaterThan	0.8	urgent
4	type	equals	analytics	batch
5	priority	equals	low	batch

🧠 Key Understanding

Concept	What you learned
TypeScript interfaces	Define the exact shape of data — compiler catches wrong field names or types before runtime
Mongoose Schema	MongoDB is schema-less by default — Mongoose adds structure and validation on top
Mixed type	Used for value field in Rule — can be string or number depending on the operator

Redis client	Single shared instance exported and reused across the application — not reconnected each time
Database seeding	Inserting known test data before development starts so the Rule Engine has rules to evaluate
mongoose.disconnect()	Must be called explicitly after writes — otherwise the process hangs open indefinitely

⚠️ Issues Faced & Fixed

Issue	Fix
db.rules.find() showed empty in Docker MongoDB	Discovered Homebrew MongoDB Community was also running on port 27017 — TypeScript was connecting to Homebrew MongoDB, not Docker. Confirmed data exists in Homebrew MongoDB and continued using it
process.exit(0) closing before MongoDB write completed	Replaced with mongoose.disconnect() which waits for all pending operations to finish before closing

📦 Files Created This Week

File	Purpose
src/models/message.model.ts	TypeScript interface for every message in the system
src/models/rule.model.ts	TypeScript interface for every routing rule
src/db/mongo.ts	MongoDB connection, Rule schema, RoutingHistory schema
src/db/redis.ts	Redis client and connection function
src/db/seedRules.ts	One-time script to populate MongoDB with 5 sample rules

📊 Database Status After Week 3

Database	Status	Contents
MongoDB (orchestrator)	✅ Connected	5 routing rules in rules collection

Redis	✅ Connected	Empty — rules will be cached here in Week 4 by Rule Engine

🎉 WEEKS 4 & 5 COMPLETE!

CI is green — Rule Engine + all 9 tests passing automatically on every push! ✅

Give me a moment and I'll give you the detailed Week 4 & 5 report, then we move to **Week 6 — the Orchestrator** which is the biggest piece of the whole system! 👇

Weeks 4 & 5 — Detailed Progress Report

🎯 Goal

Build the Rule Engine — the brain of the entire system. Load routing rules from Redis cache (or MongoDB on cache miss). Evaluate every incoming message against rules in priority order. Return urgent or batch. Write and pass all unit tests.

✅ What Was Done

1. Rule Engine Class

- Created src/ruleEngine/ruleEngine.ts
- Full class with 4 methods as per technical specification:

Method	Purpose
loadRules()	Loads rules from Redis or MongoDB into memory
evaluate()	Takes a message, returns urgent or batch
check()	Applies a single operator against a field value
invalidateCache()	Deletes Redis key and reloads fresh rules from MongoDB

2. Redis Caching Strategy

- On first call — checks Redis for key routing_rules
- Cache miss → fetches from MongoDB sorted by priority ascending
- Serialises rules to JSON → stores in Redis with 3600 second TTL
- On second call — reads directly from Redis, skips MongoDB entirely
- Cache invalidated instantly when rules are added or deleted via API

3. Four Operators Implemented

Operator	Behaviour
equals	Strict equality — case sensitive for strings

contains	Converts field to string, checks for substring
greaterThan	Converts both sides to Number before comparing
lessThan	Converts both sides to Number before comparing

4. Unit Tests

- Created src/ruleEngine/ruleEngine.test.ts
- 9 tests covering every case in the technical specification
- Tests use injected mock rules — no MongoDB or Redis needed to run
- Helper functions makeMessage() and makeEngine() keep tests clean

5. Jest Setup

- Installed jest, ts-jest, @types/jest
- Configured package.json with Jest preset for TypeScript
- npm test runs all tests locally
- CI pipeline updated to run npm test on every push

🧠 Key Understanding

Concept	What you learned
Redis caching	First rule load hits MongoDB once — every message after reads from RAM in under 1ms
Cache invalidation	When a rule changes, old cache is deleted and fresh rules loaded — takes effect on next message
Priority order	Rules sorted ascending by priority number — rule 1 evaluated before rule 2, first match wins
Unit testing	Tests run with mock data — no real database needed, fast and reliable
ts-jest	Lets Jest understand TypeScript directly — no need to compile first
CI testing	Every push to GitHub now automatically runs all 9 tests — broken code is caught immediately

🧪 All 9 Test Results

Test	Result
Critical priority → urgent	✅ Pass
Amount > 10000 → urgent	✅ Pass
RiskScore > 0.8 → urgent	✅ Pass
Analytics type → batch	✅ Pass

Low priority → batch	✅ Pass
No rule match → batch default	✅ Pass
Contains operator substring match	✅ Pass
Priority order — first match wins	✅ Pass
Missing optional fields — no crash	✅ Pass

⚠️ Issues Faced & Fixed

Issue	Fix
Duplicate "test" key in package.json	Removed the old echo test script, kept only "test": "jest"

📦 Files Created This Week

File	Purpose
src/ruleEngine/ruleEngine.ts	Core Rule Engine class — loads rules, evaluates messages, manages cache
src/ruleEngine/ruleEngine.test.ts	9 unit tests covering all operators and edge cases

📊 Progress So Far

Week	What	Status
Week 1	Docker + Kafka flow	✅ Done
Week 2	GitHub + CI pipeline	✅ Done
Week 3	Data models + MongoDB + Redis + Seeding	✅ Done
Week 4 & 5	Rule Engine + Unit Tests	✅ Done
Week 6 & 7	Orchestrator + Retry + DLQ	➡️ SOON Next

🎉 WEEKS 6 & 7 COMPLETE!

Code is on GitHub ✅

📊 Full Progress — Weeks 1 to 7

Week	What	Status
Week 1	Docker + Kafka flow	✅ Done
Week 2	GitHub + CI pipeline	✅ Done
Week 3	Data models + MongoDB + Redis + Seeding	✅ Done

Week 4 & 5	Rule Engine + 9 Unit Tests	✅ Done
Week 6 & 7	Orchestrator + Retry + DLQ + History	✅ Done

That's exactly 40% of the project — perfectly on track for your mid evaluation! 🎯

🧠 What the system can do RIGHT NOW

Producer sends message → Kafka receives it

↓

Orchestrator reads it → Rule Engine evaluates in <1ms

↓

urgent or batch decision made

↓

Published to correct RabbitMQ queue

↓

Routing history saved to MongoDB

↓

If RabbitMQ fails → retries 2s, 4s, 8s, 16s → DLQ

Do you want the **detailed Week 6 & 7 report** before we update the PPT? Or straight to updating the PPT to show 40% progress? 👇